minidb

A Mini SQL Database Engine in Python

Project Report

A thread-safe, crash-safe relational engine
with a Python DB-API 2.0 interface

Devansh Abhay Dhok, Bhavya Chauhan, Chaudhari Divyesh, Anjali Patra, Vihaan Sapra

April 2026

Abstract

minidb is a small relational database engine written entirely in Python. It supports a useful subset of SQL, exposes a Python DB-API 2.0 (PEP 249) compatible interface, and provides two properties that are non-trivial to combine: thread-safe concurrent access through a Read-Copy-Update (RCU) storage layer, and crash-safe durability through a write-ahead log with idempotent recovery. This report describes the engine's features, architecture, internal modules and functions, supported SQL syntax, Python API, transaction and concurrency models, testing methodology, and instructions for running it.

Contents

1 Introduction **3**
1.1 What minidb is . . . 3
1.2 Design goals . . . 3

2 Feature Overview **3**
2.1 SQL surface . . . 3
2.2 Python API . . . 3
2.3 Engine guarantees . . . 3

3 Architecture **4**
3.1 File layout . . . 4

4 Module Reference **4**
4.1 `minidb/__init__.py` . . . 4
4.2 `exceptions.py` . . . 5
4.3 `types.py` . . . 5
4.4 `catalog.py` . . . 5
4.5 `index.py` . . . 5
4.6 `storage.py` . . . 5
4.7 `wal.py` . . . 5
4.8 `parser.py` . . . 6
4.9 `engine.py` . . . 6
4.10 `api.py` . . . 6

5 SQL Reference **7**
5.1 Data definition . . . 7
5.2 Data manipulation . . . 7
5.3 Queries . . . 7
5.4 Transactions in SQL . . . 7
5.5 Operators and patterns . . . 7

6 Python API **8**
6.1 Connecting . . . 8
6.2 Executing statements . . . 8
6.3 Reading results . . . 8
6.4 Transactions . . . 8
6.5 Errors . . . 9

7 Transactions, Durability, and Recovery **9**
7.1 The transaction model . . . 9

7.2 Commit ordering . . . 9
7.3 Rollback . . . 9
7.4 DDL is auto-commit . . . 9
7.5 Recovery on connect . . . 9

8 Concurrency **10**
8.1 The reader path . . . 10
8.2 The writer path . . . 10
8.3 The dirty flag . . . 10

9 How to Run **10**
9.1 Requirements . . . 10
9.2 Setup . . . 10
9.3 Run the tests . . . 10
9.4 Run the demo . . . 11
9.5 Use it from your own code . . . 11

10 Testing **11**
10.1 Functional tests . . . 11
10.2 Concurrency tests . . . 11
10.3 Results . . . 12

11 Limitations and Future Work **12**

12 Conclusion **12**

1 Introduction

1.1 What minidb is

minidb is a single-file-per-table relational database. Each connection opens a directory on disk; each table is stored as its own JSON-line data file in that directory. SQL is parsed in-process, executed against an in-memory representation of the data, and persisted to disk on commit through a write-ahead log.

The engine is intentionally compact – the entire package is around 1,500 lines of Python – but it implements the features that matter for a real workload: a usable SQL surface, transactions with rollback, crash recovery, multi-threaded access, primary-key indexing, and a standard Python driver-style API.

1.2 Design goals

- **Correctness over cleverness.** Every commit is durable, every rollback is faithful, and every crash leaves the database in a consistent state.
- **Familiar API.** A user who knows `sqlite3` can use `minidb` without reading the manual.
- **Real concurrency.** Many readers and many writers can share a connection-pool style workload from multiple threads without corrupting one another's view of the data.
- **Small and readable.** Every module is short enough to fit in the head all at once.

2 Feature Overview

2.1 SQL surface

- `CREATE TABLE [IF NOT EXISTS]` with column types `INTEGER`, `TEXT`, `REAL`, `PRIMARY KEY`, and `NOT NULL`.
- `DROP TABLE [IF EXISTS]`.
- `INSERT`, `UPDATE`, `DELETE` with parameter binding.
- `SELECT` with column lists or `*`, `WHERE` (using `=`, `!=`, `<>`, `<`, `<=`, `>`, `>=`, `LIKE` with `AND`/`OR`), `ORDER BY` (multi-column, ASC/DESC, NULLs first), `LIMIT`, `DISTINCT`, and `COUNT(*)`.
- `LIKE` pattern matching with `%` (any sequence) and `_` (single character) wildcards.
- Explicit `BEGIN`, `COMMIT`, and `ROLLBACK` as SQL statements.

2.2 Python API

A complete PEP 249 surface: `connect()`, `Connection`, `Cursor`, `execute`, `executemany`, `fetchone`, `fetchmany`, `fetchall`, iteration over the cursor, context-manager support on the connection, the standard exception hierarchy, and module-level `paramstyle = "qmark"`.

2.3 Engine guarantees

- **ACID transactions** per connection.
- **Read-your-own-writes** – a `SELECT` after an `INSERT` in the same transaction sees its own pending changes.
- **Crash recovery** on `connect()` replays committed transactions and discards uncommitted ones.
- **Hash index** on every `PRIMARY KEY` column, used automatically for equality lookups.
- **Thread-safe** concurrent reads and writes through RCU.

3 Architecture

The engine is organised as a thin stack of cooperating modules. A SQL string travels through the parser, then the engine, and ultimately reaches the storage layer; a commit additionally walks the redo log through the WAL and out to disk.

```
+-----------------------+    user calls execute(),
|  PEP 249 API          | <- fetchall(), commit(), ...
|  api.py               |
+-----------+-----------+
            |
            v
+-----------------------+    +----------------------+
|  SQL Engine           | -> |  Parser              |
|  engine.py            |    |  parser.py           |
+-----------+-----------+    +----------------------+
            |
            v
+-----------------------+    +----------------------+
|  RCU Storage          | <- |  Catalog & Index     |
|  storage.py           |    |  catalog.py, index.py|
+-----------+-----------+    +----------------------+
            |
            v
       data files                Write-Ahead Log
       (per table)                  wal.py
```

3.1 File layout

```
minidb/
  __init__.py    package exports + DB-API attributes
  exceptions.py  PEP 249 exception hierarchy
  types.py       INTEGER / TEXT / REAL with coercion
  catalog.py     Schema and Catalog persistence
  index.py       hash index used for primary-key lookups
  parser.py      tokeniser + recursive-descent parser -> AST
  storage.py     RCU thread-safe Table with dirty-flag flush
  wal.py         append-only JSON-line write-ahead log
  engine.py      SQL planner and executor
  api.py         Connection / Cursor (PEP 249 surface)
tests/
  test_main.py        51 functional tests
  test_concurrency.py 11 concurrency tests
demo.py          end-to-end demo of every feature
README.md
```

4 Module Reference

This section walks through each module and lists its key classes and functions.

4.1 `minidb/__init__.py`

Exposes the public surface of the package. The single user-facing function is:

- `connect(database)` – open or create a database directory and return a `Connection`.

Module-level constants follow PEP 249: `apilevel = "2.0"`, `threadsafety = 2`, `paramstyle = "qmark"`.

4.2 exceptions.py

The standard PEP 249 exception hierarchy: `Error`, `InterfaceError`, `DatabaseError`, `DataError`, `OperationalError`, `IntegrityError`, `InternalError`, `ProgrammingError`, `NotSupportedError`.

4.3 types.py

Defines the three column types and coercion rules.

- `coerce(value, type_name)` – convert and validate a value against a column type. Raises `DataError` on a type mismatch.

4.4 catalog.py

Persistent table metadata.

- `Schema(name, columns)` – a single table's column list, types, primary key, and `NOT NULL` flags.
  - `column_names()`, `type_of(col)`, `has_column(col)`
  - `is_not_null(col)` – whether `col` carries the `NOT NULL` constraint.
  - `primary_key()` – the PK column name, or `None`.
  - `to_dict()` / `from_dict(d)` – JSON serialisation.
- `Catalog(db_dir)` – collection of `Schema` objects persisted to `catalog.json`.
  - `add(schema)`, `drop(name)`, `get(name)`, `save()`.

4.5 index.py

A primary-key hash index.

- `HashIndex(column)` – maps a PK value to a row id. `find`, `contains`, `insert`, `remove`, `rebuild(rows)`.

The engine consults the index for every `WHERE pk = ?` lookup, turning what would otherwise be a full scan into an O(1) operation.

4.6 storage.py

The thread-safe table store.

- `Table(db_dir, name)` – a single table on disk plus its in-memory representation.
  - `insert(row)`, `update(row_id, new_row)`, `delete(row_id)` – mutations under the per-table write lock.
  - `get(row_id)`, `scan()`, `live_rows()` – reads via an RCU snapshot.
  - `flush()` – write to disk and `fsync`, but only if the dirty flag is set.
  - `reload()` – rebuild in-memory state from disk.
- Internal RCU helpers: `_acquire_read`, `_release_read`, `_synchronize_rcu(old_rows)`. See §8 for how they fit together.

4.7 wal.py

An append-only JSON-line write-ahead log.

- `WAL(db_path)`

  - `log(record)` – append one record.
  - `flush()` – `fsync` the log.
  - `read_all()` – read every record, tolerating a torn final line by truncating it.
  - `clear()` – truncate after a successful commit.
  - `close()` – close the file handle.

4.8 parser.py

Tokeniser plus recursive-descent parser. The output is a tree of typed dataclass nodes:

- Statement nodes: `CreateTable`, `DropTable`, `Insert`, `Select`, `Update`, `Delete`, `Begin`, `Commit`, `Rollback`.
- Helper nodes: `ColumnDef`, `OrderBy`, `Comparison`, `BoolOp`, `Placeholder`.
- `tokenize(sql)` – produce a list of `(kind, value)` tokens.
- `parse(sql)` – the public entry point; returns the AST.

4.9 engine.py

Where SQL actually runs.

- `Database(db_dir)` – owns the catalog, the open `Table` objects, and the global commit/reload mechanics. `commit()` flushes every dirty table; `reload_from_disk()` is used during recovery.
- `Engine(db, connection)` – per-connection executor.
  - `execute(ast, params)` – the dispatch entry point.
  - `_create_table(ast)` / `_drop_table(ast)` – DDL handlers (auto-commit).
  - `_insert(ast, params)`, `_update(ast, params)`, `_delete(ast, params)` – DML handlers; each applies the change to storage immediately and records an undo and a redo entry on the connection.
  - `_select(ast, params)` – evaluates a query, applying `DISTINCT`, `ORDER BY`, `LIMIT`, and `COUNT(*)`.
  - `_iter_matching_rows` / `_pk_equality_lookup` – routes a `WHERE` to either an index lookup or a scan.
  - `_matches(row, where, params)` – evaluates a `WHERE` condition.
- Module helpers: `_sort_key` (NULL-aware sort key for `ORDER BY`), `_like_match` (translates SQL `LIKE` into a regex), `_apply_op` (single comparison evaluator).

4.10 api.py

The PEP 249 surface and the orchestration of WAL + undo.

- `Connection(database)` – opens (or creates) the database directory and runs recovery.
  - `cursor()` – create a `Cursor`.
  - `commit()` – write redo log + COMMIT to the WAL, fsync, flush data files, truncate the WAL.
  - `rollback()` – walk the in-memory undo log in reverse to restore prior state.
  - `close()` – roll back any in-flight transaction and release resources (does *not* auto-commit).
  - `__enter__` / `__exit__` – context-manager support: commit on clean exit, roll back on exception.
  - `_record_change(undo, redo)` – internal helper used by the engine.
  - `_recover()` – called once on connect; replays committed WAL transactions and discards uncommitted tails.
  - `_apply_redo_safe(record)` / `_undo_apply(record)` – idempotent appliers used by recovery and rollback.

- `Cursor(connection)`
  - `execute(sql, params=())` – parse and execute one statement.
  - `executemany(sql, seq_of_params)` – repeat `execute` over a sequence.
  - `fetchone()`, `fetchmany(size=None)`, `fetchall()` – result-set consumers.
  - `__iter__` / `__next__` – cursors are iterators.
  - `description`, `rowcount`, `arraysize`, `close()`.

5 SQL Reference

5.1 Data definition

```sql
CREATE TABLE [IF NOT EXISTS] users (
    id    INTEGER PRIMARY KEY,
    name  TEXT NOT NULL,
    email TEXT,
    age   INTEGER
);

DROP TABLE [IF EXISTS] users;
```

5.2 Data manipulation

```sql
INSERT INTO users (id, name, age) VALUES (?, ?, ?);

UPDATE users SET age = ? WHERE id = ?;

DELETE FROM users WHERE age < ?;
```

5.3 Queries

```sql
SELECT * FROM users;

SELECT name, email FROM users
 WHERE email LIKE ?
 ORDER BY name ASC
 LIMIT 10;

SELECT DISTINCT status FROM orders;

SELECT COUNT(*) FROM users WHERE age >= ?;
```

5.4 Transactions in SQL

```sql
BEGIN;
INSERT INTO users VALUES (?, ?, ?, ?);
COMMIT;     -- or  ROLLBACK;
```

5.5 Operators and patterns

- Comparison: `=`, `!=`, `<>`, `<`, `<=`, `>`, `>=`.
- Boolean: `AND`, `OR` (no `NOT`).
- `LIKE` wildcards: `%` matches any sequence of characters; `_` matches exactly one.

- Parameter placeholder: `?` (qmark style).

6 Python API

A short tour of the PEP 249 surface.

6.1 Connecting

```
import minidb

conn = minidb.connect("./mydata")  # creates dir if missing
cur  = conn.cursor()
```

6.2 Executing statements

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id   INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        age  INTEGER
    )
""")

cur.executemany(
    "INSERT INTO users VALUES (?, ?, ?)",
    [(1, "Alice", 30), (2, "Bob", 25), (3, "Cara", 28)],
)
conn.commit()
```

6.3 Reading results

```
cur.execute("SELECT name, age FROM users WHERE age >= ? ORDER BY age", (28,))

# any of:
row  = cur.fetchone()        # one row, or None
rows = cur.fetchmany(2)      # up to 2 rows
rows = cur.fetchall()        # all remaining rows
for row in cur:              # iterator protocol
    print(row)
```

6.4 Transactions

```
cur.execute("UPDATE users SET age = ? WHERE id = ?", (31, 1))
conn.commit()      # durable

cur.execute("DELETE FROM users WHERE id = ?", (2,))
conn.rollback()    # undone in memory

# Or via context manager: commits on clean exit, rolls back on exception.
with minidb.connect("./mydata") as conn:
    cur = conn.cursor()
    cur.execute("INSERT INTO users VALUES (?, ?, ?)", (4, "Dan", 22))
```

6.5 Errors

All errors derive from `minidb.Error`. The most useful subclasses are `IntegrityError` (PK uniqueness, NOT NULL), `ProgrammingError` (bad SQL or misuse), and `OperationalError` (recovery failure, I/O issues).

7 Transactions, Durability, and Recovery

7.1 The transaction model

Every connection has an implicit transaction. Mutations (`INSERT`, `UPDATE`, `DELETE`) are applied to the in-memory storage immediately, so a subsequent `SELECT` from the same connection sees its own changes. Each mutation also appends two records to per-connection logs:

- an **undo entry** describing how to reverse the change;
- a **redo entry** describing how to re-apply it.

7.2 Commit ordering

`commit()` performs the following steps in strict order:

1. Write each redo record to the WAL.
2. Write a `COMMIT` marker to the WAL.
3. `fsync` the WAL. *This is the durability barrier.*
4. Flush every dirty data file (each individually fsynced).
5. Truncate the WAL.
6. Clear the per-connection undo and redo logs.

A crash anywhere in steps 1–3 leaves the WAL with a missing or partial `COMMIT`; recovery will discard it. A crash in steps 4–5 leaves a complete WAL whose redo records will be replayed on the next start; because the redo handlers are idempotent, this is always safe.

7.3 Rollback

`rollback()` walks the in-memory undo log in reverse and applies each inverse operation to storage. It does not touch disk at all, so it is fast even for large transactions.

7.4 DDL is auto-commit

`CREATE TABLE` and `DROP TABLE` take effect immediately and discard any pending undo and redo entries on the connection. DDL is never rolled back.

7.5 Recovery on connect

When a connection is opened, the engine reads the WAL and groups records by transaction. Transactions that end in a `COMMIT` marker are replayed; everything else is discarded, including a torn final line. The replay calls the same idempotent `_apply_redo_safe` handlers used during normal operation:

- `INSERT_REDO` – skips if the PK is already present.
- `UPDATE_REDO` – looks up the row by old or new PK.
- `DELETE_REDO` – no-ops if the row is already gone.

After replay, data files are flushed and the WAL is truncated, leaving the database in a clean state.

8 Concurrency

minidb's storage layer uses **Read-Copy-Update** so that readers never block writers and writers never block readers; only writers block other writers, which is the minimum coordination required to keep mutations atomic.

8.1 The reader path

A reader takes the small `_rcu_lock`, copies a reference to the current row list, increments a per-snapshot reader counter, and releases the lock. From that point on it iterates the snapshot *without* holding any further locks. When the iteration finishes it decrements the counter under `_rcu_lock`.

8.2 The writer path

A writer holds the per-table `_write_lock` for the duration of its modification (this is what serialises writers). Inside that lock it:

1. copies the current row list,
2. applies its mutation to the copy,
3. takes `_rcu_lock`, swaps the new list into place, and releases the lock,
4. sets the dirty flag (still under the write lock).

After releasing the write lock it calls `_synchronize_rcu(old_rows)`, which spins (with brief sleeps) until the reader counter on the old snapshot reaches zero. Doing this *outside* the write lock is essential – it lets the next writer proceed even while readers are still draining off the old snapshot.

8.3 The dirty flag

`flush()` is a no-op for tables that have not been mutated since the last flush, which avoids redundant disk I/O. The flag is both set and cleared while holding the write lock, so a concurrent flusher can never have its bytes written before a still-running writer's bytes, nor can it clear the flag at a moment that would lose a writer's changes.

9 How to Run

9.1 Requirements

Python 3.10 or later. There are no third-party dependencies.

9.2 Setup

Unzip the project and `cd` into it:

```
$ unzip minidb_merged.zip
$ cd minidb_merged
```

9.3 Run the tests

```
$ python tests/test_main.py
.......................................................
Ran 51 tests in 0.4s
OK
```

```
$ python tests/test_concurrency.py
...........
Ran 11 tests in 5.2s
OK
```

9.4 Run the demo

```
$ python demo.py
```

The demo creates a temporary database, exercises every feature end-to-end (CRUD, ORDER BY, LIMIT, DISTINCT, COUNT, LIKE, NOT NULL, context-managed transactions, constraint violations), reopens the database to demonstrate persistence, and finally simulates a mid-transaction crash and verifies that recovery preserves only the committed row.

9.5 Use it from your own code

```python
import minidb

with minidb.connect("./mydata") as conn:
    cur = conn.cursor()
    cur.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, body
    TEXT NOT NULL)")
    cur.execute("INSERT INTO notes VALUES (?, ?)", (1, "hello"))
    cur.execute("SELECT body FROM notes WHERE id = ?", (1,))
    print(cur.fetchone())
```

10 Testing

10.1 Functional tests

`tests/test_main.py` contains 51 tests organised into seven classes:

Class	Tests	Coverage
`CRUDTest`	11	insert, update, delete, select, PK lookup
`SelectExtrasTest`	11	ORDER BY, LIMIT, DISTINCT, COUNT(*), LIKE
`ConstraintsTest`	4	PK uniqueness, NOT NULL on insert/update
`TransactionTest`	9	commit, rollback variants, BEGIN/COMMIT/ROLLBACK SQL, context man
`CrashRecoveryTest`	5	committed survives crash, uncommitted discarded
`PEP249Test`	8	executemany, fetch*, cursor iterator
`PersistenceTest`	3	data survives clean reconnect

10.2 Concurrency tests

`tests/test_concurrency.py` contains 11 tests targeting the concurrency machinery directly:

- `DirtyFlagTest` – 6 tests checking when the dirty flag is and isn't set, including no-op updates and deletes.
- `RCUSnapshotTest` – verifies the two key invariants: a writer must wait for an in-flight reader of the old snapshot, and a reader arriving after a swap sees the new state.

- `ConcurrentStressTest` – runs eight reader threads and four writer threads simultaneously for two seconds, with a concurrent flusher in the mix, and verifies no writes are lost and no exceptions are raised.

10.3 Results

All 62 tests pass. The functional suite finishes in roughly 0.4 seconds; the concurrency suite takes about 5 seconds because of the deliberate stress duration.

11 Limitations and Future Work

minidb is intentionally small. The following are not implemented and would be natural extensions:

- **Joins, subqueries, and grouped aggregates.** The parser and a small query planner would need work; the storage layer would not.
- **More aggregates.** `SUM`, `AVG`, `MIN`, `MAX`, and `GROUP BY` would slot into the existing `_select` path.
- **`ALTER TABLE`.** Schema evolution is not yet supported.
- **Multi-process safety.** The WAL is not coordinated between OS processes; intra-process threading is fully supported.
- **Secondary indexes.** Only the primary key is indexed.
- **B-tree storage.** A list-of-rows representation is fine for small tables but does not scale; a B-tree would give logarithmic non-PK lookups and better update performance for large tables.

12 Conclusion

minidb demonstrates that a usable relational engine – with a real SQL surface, a familiar Python API, ACID transactions, crash recovery, and multi-threaded concurrency – fits comfortably in around 1,500 lines of pure Python. The design keeps each concern in its own module, uses RCU to avoid the read-write lock contention that plagues naive implementations, and uses a write-ahead log with idempotent recovery to make commits genuinely durable. A test suite of 62 tests, all passing, covers the SQL surface, the transactional semantics, the crash-recovery story, and the concurrency invariants.